

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕЛЕКОММУНИКАЦИИ (ИУ3)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА**

ОТЧЕТ

по лабораторной работе № 5

Название: Ансамбли моделей машинного обучения. Часть 1

Дисциплина: Технологии машинного обучения

Студент ИУ3С-51Б (Группа) _____ (Подпись, дата) М.Ю. Гордон (И.О. Фамилия)

Преподаватель _____ (Подпись, дата) О.Г. Саргсян (И.О. Фамилия)

Москва, 2026

1. Описание задания

Цель лабораторной работы — изучение ансамблей моделей машинного обучения.

- Выберите набор данных (датасет) для решения задачи классификации или регресии.
- В случае необходимости проведите удаление или заполнение пропусков и кодирование категориальных признаков.
- С использованием метода train_test_split разделите выборку на обучающую и тестовую.
- Обучите следующие ансамблевые модели:
    - две модели группы бэггинга (бэггинг или случайный лес или сверхслучайные деревья);
    - AdaBoost;
    - градиентный бустинг.
- Оцените качество моделей с помощью одной из подходящих для задачи метрик. Сравните качество полученных моделей.

Используется датасет Mushroom Dataset, содержащий информацию о грибах. Датасет включает 8124 записи и 23 столбца: 22 признака, описывающих характеристики грибов, и один целевой признак `class`, указывающий, является гриб съедобным (`e`) или ядовитым (`p`).

Признаки содержат информацию о форме, поверхности и цвете шляпки, запахе, характеристиках пластинок, ножки, кольца, спор и среде обитания гриба. Исходные данные являются категориальными, поэтому перед обучением моделей категориальные признаки преобразуются в числовой.

2. Текст программы по ноутбукам и их результаты

```
import pandas as pd
```

```python
# Загружаем датасет
df = pd.read_csv("mushrooms.csv")

# Выводим первые 5 строк
print(df.head())

# Размер датасета
print("\nРазмер датасета:", df.shape)

# Информация о столбцах и типах данных
print("\nИнформация о датасете:")
df.info()

# Проверяем пропущенные значения
print("\nПропущенные значения:")
print(df.isnull().sum())
```

```
  class cap-shape cap-surface cap-color bruises odor gill-attachment  \
0     p         x           s         n       t    p               f
1     e         x           s         y       t    a               f
2     e         b           s         w       t    l               f
3     p         x           y         w       t    p               f
4     e         x           s         g       f    n               f

  gill-spacing gill-size gill-color  ... stalk-surface-below-ring  \
0            c         n          k  ...                        s
1            c         b          k  ...                        s
2            c         b          n  ...                        s
3            c         n          n  ...                        s
4            w         b          k  ...                        s
```

```
  stalk-color-above-ring stalk-color-below-ring veil-type veil-color  \
0                      w                      w         p          w
1                      w                      w         p          w
2                      w                      w         p          w
3                      w                      w         p          w
4                      w                      w         p          w

  ring-number ring-type spore-print-color population habitat
0           o         p                 k          s       u
1           o         p                 n          n       g
2           o         p                 n          n       m
3           o         p                 k          s       u
4           o         e                 n          a       g
```

```
Размер датасета: (8124, 23)

Информация о датасете:
<class 'pandas.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   class                     8124 non-null   str
 1   cap-shape                 8124 non-null   str
 2   cap-surface               8124 non-null   str
 3   cap-color                 8124 non-null   str
```

```
 4   bruises                  8124 non-null   str
 5   odor                     8124 non-null   str
 6   gill-attachment          8124 non-null   str
 7   gill-spacing             8124 non-null   str
 8   gill-size                8124 non-null   str
 9   gill-color               8124 non-null   str
 10  stalk-shape              8124 non-null   str
 11  stalk-root               8124 non-null   str
 12  stalk-surface-above-ring 8124 non-null   str
 13  stalk-surface-below-ring 8124 non-null   str
 14  stalk-color-above-ring   8124 non-null   str
 15  stalk-color-below-ring   8124 non-null   str
 16  veil-type                8124 non-null   str
```

```
 17  veil-color               8124 non-null   str
 18  ring-number              8124 non-null   str
 19  ring-type                8124 non-null   str
 20  spore-print-color        8124 non-null   str
 21  population               8124 non-null   str
 22  habitat                  8124 non-null   str
dtypes: str(23)
memory usage: 1.4 MB
```

```
Пропущенные значения:
class                     0
cap-shape                 0
cap-surface               0
cap-color                 0
bruises                   0
odor                      0
gill-attachment           0
gill-spacing              0
gill-size                 0
gill-color                0
stalk-shape               0
stalk-root                0
```

```
stalk-surface-above-ring    0
stalk-surface-below-ring    0
stalk-color-above-ring      0
stalk-color-below-ring      0
veil-type                   0
veil-color                  0
ring-number                 0
ring-type                   0
spore-print-color           0
population                  0
habitat                     0
dtype: int64
```

Рисунок 1 - Информация о датасете

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Разделяем признаки и целевую переменную
X = df.drop("class", axis=1)
y = df["class"]

# Кодируем категориальные признаки
```

```
X = pd.get_dummies(X, dtype=int)

# Кодируем целевую переменную:
# e — съедобный гриб, p — ядовитый гриб
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Разделяем данные на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

display(df.describe(include="all"))
print("Размер исходных данных:", df.shape)
print("Размер матрицы признаков:", X.shape)
print("Обучающая выборка:", X_train.shape)
print("Тестовая выборка:", X_test.shape)

print("\nРаспределение классов:")
print(df["class"].value_counts())
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...
count	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124	...
unique	2	6	4	10	2	9	2	2	2	12	...
top	e	x	y	n	f	n	f	c	b	b	...
freq	4208	3656	3244	2284	4748	3528	7914	6812	5612	1728	...

4 rows × 23 columns

stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population	habitat
8124	8124	8124	8124	8124	8124	8124	8124	8124	8124
4	9	9	1	4	3	5	9	6	7
s	w	w	p	w	o	p	w	v	d
4936	4464	4384	8124	7924	7488	3968	2388	4040	3148

Рисунок 2 - Описательная статистика признаков датасета

```
Размер исходных данных: (8124, 23)
Размер матрицы признаков: (8124, 117)
Обучающая выборка: (6499, 117)
Тестовая выборка: (1625, 117)

Распределение классов:
class
e    4208
p    3916
Name: count, dtype: int64
```

Рисунок 3 - Результат 2 ноутбука

```
from sklearn.ensemble import (
    RandomForestClassifier,
    ExtraTreesClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier
)
from sklearn.metrics import accuracy_score
import pandas as pd

# Создаём модели
models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
    "Extra Trees": ExtraTreesClassifier(
        n_estimators=100,
        random_state=42
    ),
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100,
        random_state=42
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100,
        random_state=42
    )
}

# Обучаем модели и оцениваем качество
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    results.append({
        "Модель": name,
        "Accuracy": accuracy
    })

# Сравниваем результаты
```

```
results_df = pd.DataFrame(results)
print(results_df.sort_values(by="Accuracy", ascending=False).to_string(index=False))
```

```
            Модель  Accuracy
     Random Forest  1.000000
       Extra Trees  1.000000
          AdaBoost  0.998769
 Gradient Boosting  0.998769
```

Рисунок 4 - Сравнение результатов обученных моделей

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
import pandas as pd

results = []

for name, model in models.items():
    # Получаем предсказания модели
    y_pred = model.predict(X_test)

    # Рассчитываем метрики качества
    results.append({
        "Модель": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, average="weighted"
        ),
        "Recall": recall_score(
            y_test, y_pred, average="weighted"
        ),
        "F1-score": f1_score(
            y_test, y_pred, average="weighted"
        )
    })

    # Выводим отчёт по каждому классу
    print(f"\n{'=' * 50}")
    print(name)
    print(classification_report(
        y_test,
        y_pred,
        labels=[0, 1],
        target_names=label_encoder.classes_,
        zero_division=0,
        digits=4
    ))

# Формируем итоговую сравнительную таблицу
results_df = pd.DataFrame(results)
```

```
print("\nСравнение моделей:")
print(results_df.round(4).to_string(index=False))
```

```
==================================================
Random Forest
              precision    recall  f1-score   support

           e     1.0000    1.0000    1.0000       842
           p     1.0000    1.0000    1.0000       783

    accuracy                         1.0000      1625
   macro avg     1.0000    1.0000    1.0000      1625
weighted avg     1.0000    1.0000    1.0000      1625
```

Рисунок 5 - Оценка качества обучения модели “Случайный лес”

```
==================================================
Extra Trees
              precision    recall  f1-score   support

           e     1.0000    1.0000    1.0000       842
           p     1.0000    1.0000    1.0000       783

    accuracy                         1.0000      1625
   macro avg     1.0000    1.0000    1.0000      1625
weighted avg     1.0000    1.0000    1.0000      1625
```

Рисунок 6 - Оценка качества обучения модели “Сверхслучайные деревья”

```
==================================================
AdaBoost
              precision    recall  f1-score   support

           e     0.9976    1.0000    0.9988       842
           p     1.0000    0.9974    0.9987       783

    accuracy                         0.9988      1625
   macro avg     0.9988    0.9987    0.9988      1625
weighted avg     0.9988    0.9988    0.9988      1625
```

Рисунок 7 - Оценка качества обучения модели “AdaBoost”

```
==================================================
Gradient Boosting
              precision    recall  f1-score   support

           e     0.9976    1.0000    0.9988       842
           p     1.0000    0.9974    0.9987       783

    accuracy                         0.9988      1625
   macro avg     0.9988    0.9987    0.9988      1625
weighted avg     0.9988    0.9988    0.9988      1625
```

Рисунок 8 - Оценка качества обучения модели “Градиентный бустинг”

```
Сравнение моделей:
            Модель  Accuracy  Precision  Recall  F1-score
     Random Forest    1.0000     1.0000  1.0000    1.0000
       Extra Trees    1.0000     1.0000  1.0000    1.0000
          AdaBoost    0.9988     0.9988  0.9988    0.9988
 Gradient Boosting    0.9988     0.9988  0.9988    0.9988
```

Рисунок 9 - Таблица сравнения моделей

3. Вывод

В ходе лабораторной работы были изучены и применены ансамблевые методы машинного обучения для классификации грибов на съедобные и ядовитые.

Данные были подготовлены к обучению, после чего построены и сравнены четыре модели: Случайный лес, Сверхслучайные деревья, AdaBoost и Градиентный бустинг. Наилучшие результаты показали Случайный лес и Сверхслучайные деревья с точностью 100%, а AdaBoost и Градиентный бустинг - 99,88%.

Таким образом, все четыре модели продемонстрировали высокое качество классификации, при этом лучшие результаты на тестовой выборке показали Случайный лес и Сверхслучайные деревья.